

Algoritmia y Estructuras de Datos Avanzadas.

Año Académico 2026-27

Práctica 1: Python básico. Medición del tiempo de ejecución. TAD Conjunto Disjunto y Componentes Conexas.

Fecha de entrega:

- **Grupo 126:** 12 de Octubre 2026 a las 23:59 horas.
- **Grupo 127:** 13 de Octubre 2026 antes del inicio de la clase.

¡Atención! Muchas de las funciones de esta práctica (y de las demás) se pasarán por un script de corrección automática. Esto quiere decir que tenéis que escribir escrupulosamente las indicaciones en lo que se refiere a nombres de funciones, parámetros, valores de retorno, ficheros, etc. Las funciones no deben escribir nada en pantalla (si ponéis algunos print para hacer depuración, no se os olvide eliminarlos antes de entregar). El fichero debe contener únicamente las importaciones necesarias y las funciones solicitadas. No debe contener código de prueba ni instrucciones que se ejecuten automáticamente al importar el módulo. No seguir estas indicaciones puede resultar en una penalización o, en el peor de los casos, en la no corrección de la práctica.

I. Python básico y tiempo de ejecución

I.A Midiendo tiempos de ejecución

Para medir el tiempo de ejecución, la manera más directa es utilizar la función `perf_counter()` de la librería `time` de Python. La medición correcta necesita ciertas precauciones, como se explica en el apéndice de esta práctica. En este apartado utilizaremos el método allí especificado para medir el tiempo de ejecución de algunas funciones que definiremos a continuación.

I.A.1 Escribir la función:

```
time_measure(f, dataprep, Nlist, Nrep=1000, Nstat=100)
```

Con los siguientes parámetros:

- **f:** una función de un parámetro cuyo tiempo de ejecución queremos medir.
- **dataprep:** función de preparación de datos. Recibe un número entero n y devuelve una estructura de “dimensión n ” aceptada por `f` como parámetro. La generación de datos mediante **dataprep** no debe incluirse en la medición temporal.

- **Nlist:** una lista de valores enteros. El tiempo de ejecución de la función se medirá para cada uno de estos valores de n .
- **Nrep:** número de repeticiones para cada medida elemental del tiempo de ejecución.
- **Nstat:** número de repeticiones con entradas distintas de la misma dimensión para la valoración estadística.

La función devuelve una lista de la forma $[(m_0, v_0), (m_1, v_1), \dots, (m_{k-1}, v_{k-1})]$ con $k = \text{len}(Nlist)$. El elemento (m_i, v_i) contiene la media y la varianza poblacional del tiempo de ejecución de f con datos de dimensión n_i .

1.A.2 Escribir la función:

`find_duplicates(lst)`

que, dada una lista de enteros `lst`, devuelve una nueva lista con los elementos que aparecen **más de una vez** en `lst` (sin duplicar elementos en la lista devuelta y manteniendo el orden de su primera repetición).

Ejemplos ilustrativos de la función `find_duplicates(lst)`

La función debe identificar los elementos que aparecen repetidos en una lista de entrada `lst` y devolver una nueva lista con dichos elementos, respetando las siguientes reglas:

1. **Orden de preservación:** Un elemento entra en la lista final en el instante exacto en que se detecta su **primera repetición** (es decir, la segunda vez que aparece).
 2. **Sin duplicados en la salida:** Cada elemento repetido debe aparecer **una sola vez** en la lista resultante, independientemente de si aparece 2, 3 o más veces en la lista original.
- **Ejemplo A (Repeticiones intercaladas):** Dada la lista `[4, 1, 2, 1, 3, 4, 1]`, los elementos repetidos son 1 y 4. El 1 se repite por primera vez en la posición 3 y el 4 en la posición 5. Por lo tanto, la salida es `[1, 4]`.
 - **Ejemplo B (Sin duplicados o con múltiples repeticiones consecutivas):** Dada la lista `[10, 20, 10, 10, 30]`, la salida es `[10]`. Si la lista no contiene ningún elemento repetido, como `[1, 2, 3]`, la función devuelve la lista vacía `[]`.

1.A.3 Implementa la función:

`has_sum_pair(par)`

que recibe una tupla $(lst, target)$ compuesta por una lista de enteros `lst` y un valor entero `target`, y devuelve `True` si existen dos elementos en posiciones distintas dentro de la lista cuya suma sea exactamente `target`, o `False` en caso contrario.

La función debe implementarse con una complejidad temporal esperada de $O(n)$ y utilizar como máximo $O(n)$ de espacio adicional mediante el uso adecuado de un conjunto (`set`) o un diccionario (`dict`).

Ejemplos ilustrativos de la función `has_sum_pair(par)`

La función recibe como parámetro una tupla $(lst, target)$ y evalúa si existen dos elementos en **posiciones distintas** de la lista `lst` cuya suma sea igual a `target`.

- **Ejemplo A (Coincidencia exitosa / Hit):** Para la tupla ([1, 4, 7, 12, 3], 11), la función devuelve True, ya que los elementos 4 y 7 forman la suma $4 + 7 = 11$.
- **Ejemplo B (Búsqueda fallida y uso del mismo índice):** Para la tupla ([5, 8, 3], 10), la función devuelve False. Nótese que aunque $5 + 5 = 10$, no se puede sumar el elemento 5 consigo mismo al no existir otro 5 en una posición distinta dentro de la lista.

I.A.4 Medición del rendimiento:

Utiliza la función `time_measure` para analizar el tiempo de ejecución de `has_sum_pair` con listas de tamaños n entre 10 y 10 000 (eligiendo pasos adecuados).

Realiza dos mediciones independientes para comparar el comportamiento del algoritmo:

1. **Caso con coincidencia (caso promedio):** Genera entradas donde **sí** exista un par que sume `target`. El algoritmo terminará tan pronto como encuentre el primer par válido (*early exit*).
2. **Caso sin coincidencia (caso peor):** Genera entradas donde **no** exista ningún par que sume `target`. El algoritmo se verá obligado a procesar los n elementos completos de la lista antes de retornar False.

*¿Cómo varía el tiempo de ejecución en ambos casos a medida que crece n ? ¿Mantiene la función una complejidad temporal de $O(n)$ en ambos escenarios? Justifica y razona tus respuestas a partir de las gráficas obtenidas en un fichero **measure.pdf**.*

I.B Algoritmos de codificación y compresión de listas

En este apartado vamos a analizar el comportamiento empírico de dos enfoques para la codificación por longitud de rachas (*Run-Length Encoding* - RLE) sobre listas de elementos repetidos.

I.B.1 Escribir la función:

```
rle_encode_naive(lst)
```

que recibe una lista `lst` y devuelve una lista de tuplas [(elem1, count1), (elem2, count2), ...] que representa la compresión RLE. Implementa la codificación RLE de forma deliberadamente **ineficiente**. Cada vez que añadas una nueva tupla (elemento, contador) al resultado, debes hacerlo concatenando explícitamente listas mediante el operador + (por ejemplo: `resultado = resultado + [(elem, count)]`).

I.B.2 Escribir la función:

```
rle_encode_optimized(lst)
```

que realice exactamente la misma tarea que `rle_encode_naive`, pero implementada de forma **óptima** aprovechando construcciones eficientes de Python. Construye la lista `resultado` añadiendo las tuplas de forma incremental mediante el método `append()` (modificación *in-place*).

Ejemplos ilustrativos del algoritmo RLE (Run-Length Encoding)

La **codificación por longitud de rachas (RLE)** es un algoritmo de compresión sin pérdidas que reemplaza subsecuencias consecutivas de elementos idénticos por una tupla (elemento, número_de_repeticiones).

Entrada: ['A', 'A', 'A', 'B', 'B', 'C', 'A']

Salida: [('A', 3), ('B', 2), ('C', 1), ('A', 1)]

La función transforma agrupaciones de elementos idénticos consecutivos en tuplas (valor, repeticiones).

Nótese que si el elemento vuelve a aparecer tras romper la racha (como la última 'A'), se trata como una nueva tupla independiente.

- **Ejemplo A: Compresión de secuencias discretas / etiquetas (Colores)**

Dada la siguiente lista de etiquetas de datos:

```
colours = ["R", "R", "R", "G", "G", "B", "R", "R", "R", "R"]
```

Al invocar `rle_encode_optimized(entrada)` o `rle_encode_naive(entrada)`, el algoritmo identifica:

1. Tres "R" consecutivas → ("R", 3)
2. Dos "G" consecutivas → ("G", 2)
3. Una sola "B" → ("B", 1)
4. Cuatro "R" consecutivas → ("R", 4)

RLE: [("R", 3), ("G", 2), ("B", 1), ("R", 4)]

- **Ejemplo B: Compresión de intensidad de píxeles (Escala de grises)**

En el procesamiento de imágenes, las líneas con regiones contiguas de color idéntico son comunes:

```
pixels = [0, 0, 0, 0, 0, 255, 255, 255, 0, 0]
```

RLE: [(0, 5), (255, 3), (0, 2)]

I.B.3 Análisis empírico:

Usa la función `time_measure` para analizar el tiempo de ejecución de estas dos funciones sobre listas aleatorias con secuencias de elementos repetidos de diferentes tamaños n . Los valores de `Nlist` se elegirán de manera que se aprecie con claridad la diferencia entre ambas implementaciones.

*¿Cuál es el comportamiento temporal de ambas funciones al hacer crecer n ? ¿Qué impacto tienen las operaciones internas de creación y modificación de listas en Python? Justifica tus observaciones. Incluye gráficas y justificaciones en un fichero **measure.pdf**.*

II. TAD Conjunto Disjunto y Componentes Conexas

II.A TAD Conjunto Disjunto

Vamos a implementar un conjunto disjunto (CD) S sobre un conjunto universal $\{0, 1, \dots, n-1\}$ con n índices utilizando como estructura de datos un array o bien de padres o bien de rangos negativos según se ha descrito en clase.

II.A.1 Escribir la función:

```
init_cd(n: int) -> np.ndarray
```

que devuelve un array con valores -1 en las posiciones $\{0, 1, \dots, n-1\}$.

II.A.2 Escribir la función:

```
union(rep_1: int, rep_2: int, p_cd: np.ndarray) -> int
```

que devuelve el representante del conjunto obtenido como la unión por rangos de los representados por los índices rep_1, rep_2 en el CD almacenado en el array p_cd .

II.A.3 Escribir la función:

```
find(ind: int, p_cd: np.ndarray) -> int
```

que devuelve el representante del índice ind en el CD almacenado en p_cd realizando compresión de caminos.

Aunque eficaz, nuestra implementación de CD hace difícil identificar visualmente los subconjuntos de un CD.

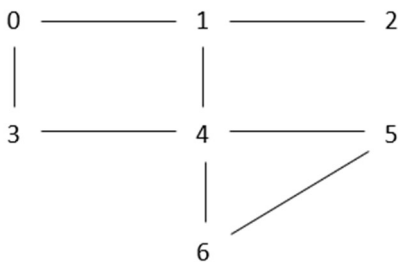
II.A.4 Escribir la función:

```
cd_2_dict(p_cd: np.ndarray) -> Dict
```

que reciba un CD en el array p_cd y devuelva un diccionario cuyas claves sean los representantes de los subconjuntos del CD y donde el valor de la clave u del dict sea una lista con los miembros del subconjunto representado por u , incluyendo, por supuesto el propio u .

II.B. CDs y Componentes Conexas

Una aplicación del TAD CD es encontrar las componentes conexas de grafos no dirigidos. Para nosotros un tal grafo va a ser un conjunto de vértices V y de ramas E tal y como se muestra en la figura inferior:



Hay diversas formas de definir una estructura de datos para grafos no dirigidos. Aunque no es muy eficaz, vamos a trabajar simplemente con un `int n` que nos indica que V va a venir dado por la lista $[0, \dots, n-1]$ y una lista l con las ramas.

Por ejemplo, para el grafo anterior $n=7$ y la lista podría ser

`[(0, 1), (0, 3), (1, 2), (1, 4), (3, 4), (4, 5), (4, 6), (5, 6)]`

Un grafo se dice **conexo** si hay un camino entre cualesquiera dos vértices. A su vez, las **componentes conexas** (CCs) de un grafo son sus subgrafos conexos maximales. En el grafo anterior solo hay una CC pero, por ejemplo, en el caso extremo de un grafo con 5 vértices y una lista vacía de ramas, habría 5 CCs, una para cada nodo. Queremos desarrollar una función que reciba un grafo no dirigido con la EdD anterior y nos devuelva un dict con sus CCs según describimos a continuación.

II.B.1 Escribir la función:

`ccs(n: int, l: List)-> Dict`

que nos devuelva las componentes conexas de un tal grafo. Para ello la función inicializará un CD vacío, examinará las ramas del grafo contenidas en la lista l y si los dos nodos de la rama pertenecen a subconjuntos distintos, unirá estos. Cuando se procesen todas las ramas la función convertirá la tabla que contenga el CD en un dict y devolverá éste.

II.C. Cuestiones sobre CDs y CCs

Contestar razonadamente a las siguientes cuestiones.

1. Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?
2. Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que a su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.
3. El tamaño de un grafo no dirigido viene determinado por el número n de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.

4. III. ¿Qué tengo que entregar?

i) Un fichero **p1.py** que contenga:

- Las funciones solicitadas en el apartado I.A (`time_measure`, `find_duplicates`, `has_sum_pair`) y en el apartado I.B (`rle_encode_naive`, `rle_encode_optimized`).
- Las funciones del apartado II (CDs y CCs).

¡Atención! El fichero debe contener sólo las funciones, ningún script de prueba.

ii) Un fichero **measure.pdf** con las gráficas solicitadas del tiempo de ejecución de las funciones (apartados I.A e I.B) y las respuestas a todas las cuestiones planteadas en el enunciado (apartados I.A, I.B y II.C).
